

Material de Estudio: Introducción al Entorno .NET y Fundamentos de C#

Profesor: Jorge Izaguirre **Materia:** Programación .NET **Curso:** 6to Año - TÉCNICO EN PROGRAMACIÓN

Introducción: El Mundo de la Programación y .NET

- **¿Qué es Programar?** En esencia, programar es darle instrucciones precisas a una computadora para que realice una tarea específica. Es como escribir una receta muy detallada en un lenguaje que la máquina pueda entender. Nosotros, los programadores, somos los "traductores" entre una necesidad humana y la lógica que una computadora puede ejecutar.
- **¿Qué es .NET?** No es un lenguaje de programación en sí mismo, sino una *plataforma* de desarrollo de software creada por Microsoft. Imagínala como una gran caja de herramientas y un taller completo. Proporciona:
 - Un *entorno de ejecución* (el taller donde se "fabrican" y "operan" los programas).
 - Una vasta *biblioteca de clases* (las herramientas prefabricadas para tareas comunes).
 - Soporte para varios lenguajes de programación (C#, F#, Visual Basic .NET).
- **¿Por qué C# (C Sharp)?** C# es el lenguaje principal y más popular para desarrollar en la plataforma .NET. Es un lenguaje moderno, potente, versátil y *orientado a objetos* (un paradigma que aprenderemos más adelante y que es fundamental en el desarrollo de software actual).
 - *¿Por qué elegir .NET y C#?*
 - **Versatilidad:** Puedes crear aplicaciones web, de escritorio, móviles, servicios en la nube, videojuegos, aplicaciones de inteligencia artificial y mucho más.
 - **Productividad:** Su amplia biblioteca y herramientas como Visual Studio aceleran el desarrollo.
 - **Comunidad y Soporte:** Existe una enorme comunidad de desarrolladores y abundante documentación y ayuda online (¡y de Microsoft!).
 - **Rendimiento:** .NET es conocido por su buen rendimiento.
 - **Multiplataforma:** Con .NET Core (y ahora .NET 5, 6, 7, 8...), tus aplicaciones C# pueden correr en Windows, macOS y Linux.

Preparando el Entorno: Visual Studio

- **¿Qué es Visual Studio (VS)?** Es un *Entorno de Desarrollo Integrado* (IDE - Integrated Development Environment).
 - *¿Por qué usar un IDE?* Porque integra en un solo lugar todo lo que necesitas para programar eficientemente:
 - Un *editor de código* inteligente (con resaltado de sintaxis, autocompletado, etc.).
 - Un *compilador* (traduce tu código C# a un lenguaje intermedio que .NET entiende).
 - Un *depurador* (herramienta crucial para encontrar y corregir errores).
 - Herramientas para gestionar proyectos, conectarse a bases de datos, y mucho más.
- **Instalación:** Usaremos la versión *Visual Studio Community*, que es gratuita para estudiantes y desarrolladores individuales. La instalación se realiza descargando el "Visual Studio Installer" desde el sitio oficial de Microsoft. Durante la instalación, es crucial seleccionar la carga de trabajo ".NET desktop development" y/o "ASP.NET and web development" para tener todo lo necesario para C# y desarrollo web/console.
- **Interfaz Principal (Un Vistazo Rápido):**

- **Explorador de Soluciones (Solution Explorer):** A la derecha (usualmente). Muestra la estructura de tu proyecto: archivos de código (.cs), dependencias, propiedades. Una *Solución* puede contener uno o más *Proyectos*.
- **Editor de Código:** El área central donde escribirás tu código C#.
- **Lista de Errores (Error List):** Abajo. Muestra errores de compilación (problemas que impiden que el programa se construya) y advertencias.
- **Ventana de Salida (Output Window):** Abajo. Muestra mensajes del proceso de compilación y, a veces, salida del programa durante la ejecución o depuración.

El Ecosistema .NET por Dentro (Conceptos Clave)

- **El Lenguaje Intermedio (CIL / MSIL):** Cuando compilas tu código C# (o VB.NET, F#), no se convierte directamente a código máquina específico de tu procesador. Se traduce a un *Lenguaje Intermedio Común* (Common Intermediate Language - CIL, antes llamado *Microsoft Intermediate Language* - MSIL).
 - *¿Por qué este paso intermedio?* Permite que diferentes lenguajes .NET puedan interactuar entre sí y hace que el código sea más portable entre diferentes arquitecturas (siempre que tengan el CLR).
- **El Entorno de Ejecución (CLR):** El *Common Language Runtime* es el corazón de .NET. Es el agente que realmente gestiona y ejecuta tu código CIL.
 - *¿Qué hace el CLR y por qué es importante?*
 - **Compilación Just-In-Time (JIT):** Convierte el CIL a código máquina nativo justo antes de ejecutarlo por primera vez en una máquina específica. Esto optimiza el rendimiento.
 - **Administración de Memoria (Garbage Collector - GC):** Libera automáticamente la memoria que tus objetos ya no usan. Esto evita muchos errores comunes en otros lenguajes donde el programador debe manejar la memoria manualmente (¡es una gran ayuda!).
 - **Seguridad:** Aplica políticas de seguridad al código.
 - **Manejo de Excepciones:** Proporciona un sistema unificado para manejar errores inesperados durante la ejecución.
 - **Interoperabilidad:** Facilita la comunicación entre código escrito en diferentes lenguajes .NET.
- **La Biblioteca de Clases Base (BCL):** Es una enorme colección de código preescrito y reutilizable que viene con .NET. Contiene miles de *clases* y *tipos* que te permiten realizar tareas comunes sin tener que programarlas desde cero.
 - *¿Por qué es tan útil?* Imagina tener que escribir el código para leer un archivo, conectarte a internet, manipular texto, o usar estructuras de datos complejas cada vez que lo necesitas. ¡Sería muy ineficiente! La BCL te da todo eso y mucho más, organizado en *espacios de nombres* (namespaces) como *System*, *System.IO*, *System.Collections*, etc. Usar la BCL acelera enormemente el desarrollo.

Tu Primer Programa en C#: "Hola, Mundo!"

1. Abre Visual Studio.
2. Crea un nuevo proyecto: Elige "Aplicación de Consola" (Console App) utilizando C#. Asegúrate de seleccionar una versión reciente de .NET (ej. .NET 6, .NET 7, .NET 8 o superior).
3. Visual Studio generará una estructura básica. En versiones recientes de .NET, el archivo *Program.cs* puede ser muy simple:

```
// Esto es un comentario de una línea

/* Esto es un comentario
   de múltiples líneas */

// La siguiente línea imprime texto en la consola
Console.WriteLine("Hola, Mundo!");
```

En versiones un poco más antiguas, podrías ver algo más estructurado:

```
using System; // Importa el namespace System (donde está Console)

namespace MiPrimeraApp // Agrupa el código en un espacio de nombres
{
    class Program // Define una clase llamada Program
    {
        // El método Main es el punto de entrada de la aplicación
        static void Main(string[] args)
        {
            Console.WriteLine("Hola, Mundo!"); // Imprime en consola
        }
    }
}
```

- **Anatomía Básica:**

- *using System;*: Indica que usaremos clases del *namespace* (espacio de nombres) *System*. Piensa en los *namespaces* como carpetas para organizar clases. *Console* vive dentro de *System*.
- *namespace MiPrimeraApp*: Define nuestro propio "espacio" para organizar nuestras clases y evitar conflictos de nombres con otras librerías.
- *class Program*: En C#, todo el código ejecutable debe estar dentro de una *clase*. Una clase es una plantilla para crear objetos (lo veremos más adelante). Por convención, la clase principal de una app de consola se llama *Program*.
- *static void Main(string[] args)*: Este es el *método* principal, el **punto de entrada** de tu programa.
 - *static*: Significa que el método pertenece a la clase *Program* en sí, no a una instancia específica de ella. Esto permite que el CLR lo llame sin tener que crear un objeto *Program* primero.
 - *void*: Indica que este método no devuelve ningún valor al sistema operativo cuando termina.
 - *Main*: Es el nombre estándar que .NET busca para empezar a ejecutar.
 - *(string[] args)*: Define un parámetro llamado *args* que es un *array* (colección) de *strings* (texto). Aquí es donde llegarían los argumentos si ejecutaras tu programa desde la línea de comandos pasándole datos extras (ej: *MiPrograma.exe archivo.txt*).
- *Console.WriteLine("Hola, Mundo!");*: Llama al método *WriteLine* de la clase *Console* para imprimir el texto "Hola, Mundo!" seguido de un salto de línea en la consola.
- *{ }*: Las llaves definen bloques de código (el cuerpo de un *namespace*, de una *clase*, de un *método*, etc.).

- `;`: El punto y coma marca el final de una instrucción en C#. Es obligatorio.
- **Ejecución:** Presiona F5 o el botón "Play" (Iniciar) en Visual Studio. VS compilará el código (lo traducirá a CIL) y luego el CLR lo ejecutará (vía JIT). Verás aparecer una ventana de consola mostrando "Hola, Mundo!".

Fundamentos de C#: Variables, Tipos y Operadores

- **Variables:** Son nombres simbólicos que usamos para almacenar datos en la memoria de la computadora. Piensa en ellas como cajas etiquetadas donde guardas información que tu programa necesita usar o modificar.
 - *¿Cómo se usan?* Primero se *declaran* (especificando el tipo de dato y el nombre) y luego se les *asigna* un valor (o se hace en un solo paso: inicialización).
 - Ejemplo:

```
int edad; // Declaración de una variable llamada 'edad' de tipo entero
edad = 30; // Asignación de valor

string nombre = "Jorge"; // Declaración e inicialización en un paso

Console.WriteLine(nombre); // Usar la variable para imprimir su valor
```

- **Tipos de Datos:** Definen qué tipo de información puede almacenar una variable y qué operaciones se pueden realizar con ella. C# es un lenguaje *fuertemente tipado*, lo que significa que una vez que declaras una variable de un tipo, no puedes asignarle un valor de un tipo incompatible directamente.
 - *¿Por qué tipos?* Ayudan a prevenir errores, permiten al compilador optimizar el uso de memoria y aclaran la intención del código.
 - **Tipos Primitivos Comunes:**
 - *int*: Números enteros (ej: -5, 0, 25). Ocupa 32 bits.
 - *double*: Números con decimales (ej: -3.14, 0.0, 99.99). Ocupa 64 bits. Es el tipo por defecto para literales decimales (si escribes **3.14**, C# asume que es **double**). Para finanzas se suele usar **decimal** por mayor precisión.
 - *bool*: Valores lógicos: *true* o *false*. Fundamental para tomar decisiones.
 - *char*: Un único carácter Unicode (ej: 'A', '?', '#'). Se escribe entre comillas simples.
 - *string*: Una secuencia de caracteres (texto). Se escribe entre comillas dobles (ej: "Hola Profe"). Aunque parece simple, es un tipo más complejo (*reference type*).
 - **Value Types vs. Reference Types (Introducción):**
 - *Value Types* (Tipos de Valor: *int*, *double*, *bool*, *char*, structs, enums): La variable contiene directamente el valor. Cuando asignas una variable de valor a otra, se crea una *copia* del valor.
 - *Reference Types* (Tipos de Referencia: *string*, arrays, clases): La variable no contiene el dato en sí, sino una *referencia* (como una dirección de memoria) a donde está almacenado el dato (el objeto). Cuando asignas una variable de referencia a otra, ambas variables apuntan al *mismo* objeto en memoria. Cambiar el objeto a través de una variable afectará a la otra. (*string* es un caso especial porque es inmutable, pero se comporta como tipo de referencia en otros aspectos). *¿Por qué importa?* Afecta cómo se pasan los datos a los métodos y cómo se gestiona la memoria.

- **Operadores:** Símbolos que realizan operaciones sobre uno o más valores (operandos).
 - **Aritméticos:**
 - `+` Suma
 - `-` Resta
 - `*` Multiplicación
 - `/` División (Ojo: división entre enteros trunca decimales. Ej: `5 / 2` da `2`. Para obtener `2.5`, al menos uno debe ser decimal: `5.0 / 2`).
 - `%` Módulo (Resto de la división entera. Ej: `5 % 2` da `1`).
 - **Asignación:**
 - `=` Asigna el valor de la derecha a la variable de la izquierda (`edad = 30`).
 - Compuestos: `+=`, `-=`, `*=`, `/=`, `%=` (Ej: `contador += 1`; es lo mismo que `contador = contador + 1`).
 - Incremento/Decremento: `++` (incrementa en 1), `--` (decrementa en 1). Pueden ir antes (`++x`) o después (`x++`), con diferencias sutiles en el valor que devuelve la expresión.
 - **Comparación:** Comparan dos valores y devuelven un *bool* (*true* o *false*).
 - `==` Igual a
 - `!=` Distinto de
 - `<` Menor que
 - `>` Mayor que
 - `<=` Menor o igual que
 - `>=` Mayor o igual que
 - **Lógicos:** Combinan expresiones booleanas.
 - `&&` AND Lógico (Y): Devuelve *true* si *ambas* expresiones son *true*. (Ej: `edad >= 18 && tieneLicencia`).
 - `||` OR Lógico (O): Devuelve *true* si *al menos una* de las expresiones es *true*. (Ej: `esFeriado || esFinDeSemana`).
 - `!` NOT Lógico (NO): Invierte el valor booleano (de *true* a *false* y viceversa). (Ej: `!estaLloviendo`).
 - **Cortocircuito:** En `expr1 && expr2`, si `expr1` es *false*, `expr2` no se evalúa. En `expr1 || expr2`, si `expr1` es *true*, `expr2` no se evalúa. ¿Por qué? Optimiza y evita errores si `expr2` depende de `expr1`.
 - **Precedencia:** Los operadores tienen un orden de evaluación (como en matemáticas: `*` y `/` antes que `+` y `-`). Usa paréntesis `()` para forzar un orden específico y mejorar la claridad.

Entrada y Salida por Consola (Interactuando con el Usuario)

- **Salida (`Console.WriteLine` y `Console.Write`):**
 - `WriteLine`: Imprime el texto (o el valor de una variable convertido a texto) y luego mueve el cursor a la siguiente línea.
 - `Write`: Imprime el texto, pero el cursor se queda al final de lo impreso.
 - **Concatenación e Interpolación:** Formas de combinar texto fijo con variables:

```
string nombre = "Ana";
int edad = 22;

// Concatenación con +
Console.WriteLine("Nombre: " + nombre + ", Edad: " + edad);
```

```
// Interpolación de cadenas (más moderna y legible) con $
Console.WriteLine($"Nombre: {nombre}, Edad: {edad}");
```

- **Entrada (Console.ReadLine):**

- Lee *toda* la línea de texto que el usuario escribe en la consola hasta que presiona Enter.
- **¡Importante!** *ReadLine* siempre devuelve un *string*.
- ¿Qué pasa si necesito un número? Debes *convertir* (o *parsear*) el *string* recibido al tipo numérico deseado.
- **Conversión:**

```
Console.Write("Ingrese su edad: ");
string edadTexto = Console.ReadLine(); // edadTexto es string

// Convertir el string a int
int edadNumero = int.Parse(edadTexto);

Console.Write("Ingrese su altura (ej: 1.75): ");
string alturaTexto = Console.ReadLine();

// Convertir el string a double (OJO con la coma/punto decimal según
// config regional)
double alturaNumero = double.Parse(alturaTexto);

int anioNacimiento = 2025 - edadNumero; // Ahora sí podemos operar
Console.WriteLine($"Usted nació aproximadamente en {anioNacimiento}.");
```

- ¿Qué pasa si el usuario escribe "hola" cuando pedimos la edad? El método *Parse* fallará y lanzará una excepción (un error en tiempo de ejecución) que detendrá tu programa si no lo manejas. Más adelante aprenderemos formas más seguras de convertir, como *int.TryParse()*. Por ahora, asumimos que el usuario ingresa datos válidos.

Controlando el Flujo del Programa

Normalmente, el código se ejecuta línea por línea, de arriba abajo. Las estructuras de control nos permiten alterar este flujo, tomando decisiones o repitiendo bloques de código.

- **Estructuras Condicionales (Tomando Decisiones):**

- **if:** Ejecuta un bloque de código *solo si* una condición (expresión booleana) es *true*.

```
if (promedio >= 7)
{
    Console.WriteLine("¡Aprobado!");
}
```

- **if-else:** Ejecuta un bloque si la condición es *true*, y otro bloque diferente si es *false*.

```

if (temperatura > 25)
{
    Console.WriteLine("Hace calor.");
}
else
{
    Console.WriteLine("No hace tanto calor.");
}

```

- **if-else if-else**: Encadena múltiples condiciones. Se evalúan en orden. La primera que sea *true* ejecuta su bloque, y el resto se ignora. El *else* final (opcional) se ejecuta si *ninguna* de las condiciones anteriores fue *true*.

```

if (nota >= 9)
{
    Console.WriteLine("Sobresaliente");
}
else if (nota >= 7)
{
    Console.WriteLine("Aprobado");
}
else if (nota >= 4)
{
    Console.WriteLine("Regular");
}
else
{
    Console.WriteLine("Desaprobado");
}

```

- **switch**: Una alternativa al *if-else if-else* cuando comparas el valor de una *única* variable o expresión contra varios valores constantes (*case*). Suele ser más legible en esos casos.
 - *¿Cómo funciona?* Evalúa la expresión del *switch*. Salta al *case* cuyo valor coincida. Ejecuta el código de ese *case* hasta encontrar un *break*.
 - *break*: Es **obligatorio** al final de cada bloque *case* en C# para salir del *switch*. Sin él, el código "caería" al siguiente *case* (lo cual rara vez se desea y C# lo prohíbe directamente en la mayoría de los casos para evitar errores).
 - *default* (opcional): Se ejecuta si ninguno de los *case* coincide. Es como el *else* final.

```

Console.WriteLine("Elija una opción (1: Sumar, 2: Restar):");
int opcion = int.Parse(Console.ReadLine());

switch (opcion)
{
    case 1:
        Console.WriteLine("Elegió Sumar.");
        // ...código para sumar...

```

```

        break; // Salir del switch
    case 2:
        Console.WriteLine("Eligió Restar.");
        // ...código para restar...
        break; // Salir del switch
    default:
        Console.WriteLine("Opción no válida.");
        break;
}

```

- **Estructuras Repetitivas (Bucles - Haciendo las cosas muchas veces):**

- **while:** Repite un bloque de código *mientras* una condición sea *true*. La condición se verifica *antes* de cada iteración. Si la condición es *false* desde el principio, el bloque no se ejecuta ni una vez.
 - *¿Cuándo usarlo?* Cuando no sabes exactamente cuántas veces necesitas repetir, pero sí sabes bajo qué condición debes continuar. Ej: seguir pidiendo un dato hasta que sea válido.

```

int contador = 0;
while (contador < 5) // Mientras contador sea menor que 5
{
    Console.WriteLine($"El contador es: {contador}");
    contador++; // ¡Fundamental! Incrementar para eventualmente salir
del bucle
}
// Al salir, contador valdrá 5

string clave = "";
while (clave != "secreto") // Mientras la clave no sea "secreto"
{
    Console.Write("Ingrese la clave: ");
    clave = Console.ReadLine();
}
Console.WriteLine("Clave correcta!");

```

- **¡Cuidado con los Bucles Infinitos!** Si la condición de un *while* nunca se vuelve *false*, el bucle se ejecutará para siempre (o hasta que detengas el programa). Asegúrate de que algo dentro del bucle eventualmente haga que la condición cambie a *false*.
- **do-while:** Similar al *while*, pero la condición se verifica *después* de ejecutar el bloque de código. Esto garantiza que el bloque se ejecute *al menos una vez*, incluso si la condición es *false* desde el principio.
 - *¿Cuándo usarlo?* Cuando necesitas realizar la acción primero y luego decidir si repetirla. Ej: mostrar un menú y luego preguntar si el usuario quiere continuar.

```

string continuar;
do
{
    // ...código para realizar una acción...
}
while (continuar != "n");

```



```

    Console.WriteLine("Acción realizada.");

    Console.Write("¿Desea realizarla de nuevo? (s/n): ");
    continuar = Console.ReadLine();
} while (continuar == "s"); // Repetir si la respuesta es "s"

```

- **for:** Es ideal cuando sabes (o puedes calcular) *cuántas* veces quieres repetir un bloque de código. Agrupa en un solo lugar la inicialización de una variable de control, la condición para continuar y la acción de incremento/decremento de esa variable.

- **Sintaxis:** `for (inicialización; condición; iteración)`

- *inicialización:* Se ejecuta una sola vez, al principio. Típicamente declara e inicializa un contador (ej: `int i = 0`).
- *condición:* Se evalúa *antes* de cada iteración. Si es *true*, se ejecuta el cuerpo del bucle. Si es *false*, el bucle termina. (ej: `i < 10`).
- *iteración:* Se ejecuta *después* de cada iteración del cuerpo. Típicamente incrementa o decrementa el contador (ej: `i++`).

```

// Mostrar números del 0 al 4
for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"Iteración número: {i}");
}

// Calcular la suma de los primeros 10 números
int suma = 0;
for (int j = 1; j <= 10; j++)
{
    suma += j; // suma = suma + j;
}
Console.WriteLine($"La suma es: {suma}"); // Imprime 55

```

Modularizando el Código: Métodos

A medida que los programas crecen, poner todo el código dentro del método *Main* se vuelve caótico, difícil de leer, de mantener y de reutilizar. Los *métodos* (también llamados funciones o procedimientos en otros contextos) nos permiten dividir un programa grande en partes más pequeñas, lógicas y reutilizables.

- **¿Por qué usar Métodos?**

- **Reutilización:** Escribes el código una vez y lo llamas (lo usas) desde diferentes partes de tu programa tantas veces como necesites. (Principio DRY: *Don't Repeat Yourself* - No te repitas).
- **Organización y Legibilidad:** Dividen el código en bloques con nombres descriptivos, haciendo el programa más fácil de entender.
- **Mantenimiento:** Si necesitas corregir o mejorar una tarea específica, solo modificas el método correspondiente, en lugar de buscar el mismo código repetido en varios lugares.
- **Abstracción:** Puedes usar un método sin necesidad de saber *exactamente* cómo funciona por dentro, solo necesitas saber qué hace, qué datos necesita (parámetros) y qué devuelve (si algo).

- **Definición de un Método (dentro de la clase *Program*, por ahora):**

```
// Modificador Acceso | static | Tipo Retorno | NombreMetodo | (Lista de
// Parámetros)
public static void MostrarMensaje(string mensaje)
{ // Inicio del cuerpo del método
    Console.WriteLine($"Mensaje importante: {mensaje}");
} // Fin del cuerpo del método

public static int Sumar(int num1, int num2)
{
    int resultado = num1 + num2;
    return resultado; // Devuelve el valor calculado
}
```

- *Modificador de Acceso (public)*: Define desde dónde se puede llamar al método. *public* significa desde cualquier lugar. (Otros son *private*, *internal*, etc.).
- *static*: Igual que en *Main*, indica que el método pertenece a la clase, no a un objeto. Por ahora, como llamaremos a estos métodos desde *Main* (que es *static*), nuestros métodos también deben ser *static*.
- *Tipo de Retorno*: Especifica el tipo de dato que el método devolverá al lugar donde fue llamado.
 - *void*: Indica que el método *no* devuelve ningún valor. Simplemente realiza una acción.
 - *int*, *string*, *bool*, etc.: Indica que el método *debe* devolver un valor de ese tipo usando la palabra clave *return*.
- *NombreMetodo*: Un nombre descriptivo que sigue las convenciones de C# (PascalCase - primera letra de cada palabra en mayúscula, ej: *CalcularPromedio*).
- *Lista de Parámetros* (entre paréntesis *()*): Define los datos de entrada que el método necesita para trabajar. Cada parámetro tiene un tipo y un nombre. Si un método no necesita datos de entrada, los paréntesis van vacíos: *()*
- **Llamada a un Método**: Para ejecutar el código de un método, simplemente escribes su nombre seguido de paréntesis, proporcionando los valores (argumentos) para los parámetros definidos (si los hay).

```
static void Main(string[] args)
{
    MostrarMensaje("¡Empezando el programa!"); // Llamada al método void

    int a = 10;
    int b = 5;
    int sumaTotal = Sumar(a, b); // Llamada al método Sumar, guardando el
    // valor devuelto

    Console.WriteLine($"La suma de {a} y {b} es: {sumaTotal}"); // Imprime
    15

    MostrarMensaje("¡Terminando el programa!");
}

// --- Definiciones de los métodos (fuera de Main pero dentro de la clase
// Program) ---
```

```

public static void MostrarMensaje(string mensaje)
{
    Console.WriteLine($"Mensaje importante: {mensaje}");
}

public static int Sumar(int num1, int num2)
{
    int resultado = num1 + num2;
    return resultado;
}
// --- Fin de definiciones ---

```

- **Paso de Parámetros por Valor:** Cuando pasas variables de *tipos de valor* (*int*, *double*, *bool*, etc.) como argumentos a un método, el método recibe una *copia* de esos valores. Si el método modifica el valor de sus parámetros, las variables originales fuera del método *no* se ven afectadas. (Para *tipos de referencia* como arrays, el comportamiento es diferente, se pasa la referencia).

Trabajando con Colecciones: Arrays (Vectores)

A menudo necesitamos trabajar con conjuntos de datos del mismo tipo (una lista de notas, nombres de alumnos, precios de productos). Un *array* es una estructura de datos fundamental que nos permite almacenar una colección *ordenada* y de *tamaño fijo* de elementos del *mismo tipo*.

- **¿Por qué usar Arrays?** Permiten agrupar datos relacionados bajo un solo nombre y acceder a ellos fácilmente mediante un índice.
- **Declaración:** Se especifica el tipo de los elementos seguido de corchetes `[]` y el nombre del array.

```

int[] edades; // Declara un array que contendrá enteros
string[] nombres; // Declara un array que contendrá strings
double[] promedios; // Declara un array que contendrá doubles

```

- **Inicialización y Creación:** Declarar un array solo reserva el nombre. Necesitas crearlo (reservar la memoria) especificando su tamaño usando la palabra clave *new*. El tamaño de un array es fijo una vez creado.

```

edades = new int[5]; // Crea un array para 5 enteros.
                    // Los elementos se inicializan a 0 por defecto.

nombres = new string[3]; // Crea un array para 3 strings.
                       // Los elementos se inicializan a null por defecto.

// También puedes declarar y crear en un paso:
double[] notas = new double[10]; // Array para 10 notas (inicializadas a 0.0)

```

- **Inicialización con Valores:** Puedes proporcionar los valores iniciales directamente al crear el array.

```
int[] numerosPrimos = new int[] { 2, 3, 5, 7, 11 }; // Tamaño 5 inferido
string[] diasSemana = { "Lunes", "Martes", "Miércoles", "Jueves", "Viernes",
    "Sábado", "Domingo" }; // Sintaxis corta
```

- **Acceso a Elementos (Indexación):** Se accede a cada elemento del array usando su *índice* (posición) entre corchetes []. **¡Importante!** Los índices en C# (y muchos otros lenguajes) empiezan en **0**.
 - El primer elemento está en el índice 0.
 - El segundo elemento está en el índice 1.
 - ...
 - El último elemento está en el índice **tamaño - 1**.

```
int[] misNumeros = { 10, 20, 30 };

int primerNumero = misNumeros[0]; // primerNumero vale 10
int segundoNumero = misNumeros[1]; // segundoNumero vale 20

misNumeros[2] = 35; // Modificar el valor en el índice 2 (ahora es 35)

// ¡Error! Índice fuera de rango (válidos son 0, 1, 2)
// int error = misNumeros[3]; // Esto causará una excepción en tiempo de
// ejecución
```

- **Propiedad Length:** Todos los arrays tienen una propiedad *Length* que te dice cuántos elementos pueden contener (su tamaño). Es muy útil para recorrerlos.

```
double[] temperaturas = new double[24]; // Array para 24 temperaturas
Console.WriteLine($"El array puede almacenar {temperaturas.Length}
    temperaturas."); // Imprime 24
```

- **Recorriendo Arrays con Bucles (for):** El bucle *for* es perfecto para trabajar con arrays, ya que puedes usar el índice y la propiedad *Length*.

```
string[] alumnos = { "Juan", "Maria", "Pedro" };

Console.WriteLine("Lista de Alumnos:");
// Recorre desde el índice 0 hasta el índice (Length - 1)
for (int i = 0; i < alumnos.Length; i++)
{
    Console.WriteLine($"- {alumnos[i]} (en índice {i})");
}

// Ejemplo: Calcular el promedio de notas
double[] notas = { 8.5, 7.0, 9.2, 6.0 };
double sumaNotas = 0;
for (int i = 0; i < notas.Length; i++)
```

```
{
    sumaNotas += notas[i]; // Acumular la suma
}
double promedio = sumaNotas / notas.Length; // Calcular promedio
Console.WriteLine($"El promedio es: {promedio}");
```

Encontrando y Corrigiendo Errores: Depuración (Debugging)

Escribir código sin errores es casi imposible. Los errores (o *bugs*) son parte normal del proceso. La *depuración* es el proceso de encontrar y corregir esos errores. Visual Studio nos ofrece herramientas muy potentes para esto.

- **¿Por qué Depurar?** Intentar encontrar errores solo leyendo el código o llenándolo de `Console.WriteLine` puede ser muy lento e ineficaz, especialmente en programas complejos. El depurador te permite:
 - *Ejecutar el código paso a paso*, viendo exactamente qué línea se ejecuta a continuación.
 - *Inspeccionar el estado del programa* en cualquier punto: ver el valor de las variables, el resultado de las expresiones.
 - *Entender el flujo real* de ejecución, no solo el que *crees* que debería seguir.
- **Herramientas Básicas de Depuración en VS:**
 - **Puntos de Interrupción (Breakpoints):** Son marcadores que pones en una línea de código (haciendo clic en el margen izquierdo o presionando F9). Cuando ejecutas el programa en modo de depuración (presionando F5), la ejecución se *pausará* justo *antes* de ejecutar la línea con el breakpoint.
 - **Ejecución Paso a Paso:** Una vez pausado en un breakpoint:
 - **Step Over (F10):** Ejecuta la línea actual completa. Si la línea contiene una llamada a un método que tú escribiste, ejecuta ese método entero y se detiene en la siguiente línea *después* de la llamada. Útil cuando confías en que el método funciona y no quieres entrar a ver sus detalles.
 - **Step Into (F11):** Ejecuta la línea actual. Si la línea contiene una llamada a un método, *entra* dentro de ese método y se detiene en la primera línea del mismo. Útil para depurar el interior de tus propios métodos.
 - **Step Out (Shift + F11):** Si estás dentro de un método, ejecuta el resto del método actual y se detiene en la línea *después* de la llamada original a ese método. Útil para salir rápidamente de un método en el que ya no necesitas depurar.
 - **Inspección de Variables:** Mientras la ejecución está pausada:
 - **DataTips:** Simplemente deja el cursor del mouse sobre una variable en el editor de código y aparecerá una pequeña ventana mostrando su valor actual.
 - **Ventanas Locals y Autos:** Se suelen abrir automáticamente en la parte inferior. *Locals* muestra todas las variables accesibles en el ámbito actual (el método o bloque donde estás parado). *Autos* intenta mostrar las variables más relevantes para la línea actual y las anteriores/siguientes.
 - **Ventana Watch:** Te permite escribir nombres de variables o expresiones específicas que quieres monitorear constantemente. Sus valores se actualizan a medida que avanzas paso a paso.

- **¿Cómo usar el Depurador (Flujo Básico)?**

1. **Identifica el problema:** ¿Dónde crees que puede estar el error o dónde quieres entender mejor el flujo?
2. **Pon un Breakpoint:** Haz clic en el margen de la línea *antes* de donde sospechas que ocurre el problema.
3. **Inicia la Depuración:** Presiona F5. El programa se ejecutará hasta alcanzar tu breakpoint y se pausará.
4. **Inspecciona:** Mira los valores de las variables relevantes usando DataTips o las ventanas *Locals/Watch*. ¿Son los que esperabas?
5. **Avanza Paso a Paso:** Usa F10 o F11 para ejecutar línea por línea, observando cómo cambian los valores de las variables y cómo se toman las decisiones en los *if* o cómo avanzan los bucles *for/while*.
6. **Encuentra la Discrepancia:** Tarde o temprano, verás que el valor de una variable no es el esperado o que el programa toma un camino (ej: entra en un *if* que no debería) incorrecto. ¡Ahí está tu bug!
7. **Corrige el Código:** Detén la depuración (Shift+F5 o el botón Stop), corrige el error en el código fuente.
8. **Vuelve a Probar:** Quita breakpoints innecesarios (F9 de nuevo) y vuelve a ejecutar con F5 para asegurarte de que el error se ha corregido y no has introducido nuevos problemas.

Conclusión

Estos primeros pasos sientan las bases fundamentales para programar en C# y .NET. Hemos visto cómo configurar el entorno, la estructura básica de un programa, cómo manejar datos simples con variables y tipos, cómo controlar el flujo con condicionales y bucles, cómo organizar el código con métodos y cómo trabajar con colecciones básicas usando arrays. Además, hemos aprendido la importancia vital de la depuración. Dominar estos conceptos es esencial antes de avanzar hacia temas más complejos como la Programación Orientada a Objetos, el acceso a bases de datos y el desarrollo de aplicaciones web con MVC. ¡La práctica constante es la clave!

Síntesis de una oración: Este material introduce el ecosistema .NET y Visual Studio, cubre los fundamentos de la programación en C# (variables, tipos, operadores, control de flujo, métodos, arrays) y las técnicas básicas de depuración, esenciales para iniciar el desarrollo de software.

Síntesis de 5 puntos importantes:

1. .NET es una plataforma de desarrollo versátil con C# como lenguaje principal, ejecutada por el CLR y apoyada por la BCL.
2. Visual Studio es el IDE esencial para escribir, compilar y depurar código C#.
3. C# requiere manejo de tipos de datos, variables, operadores y estructuras de control (*if*, *switch*, *while*, *for*) para definir la lógica del programa.
4. Los Métodos son cruciales para organizar, reutilizar y hacer más legible el código.
5. La Depuración (Breakpoints, Step Over/Into, Inspección de variables) es una habilidad fundamental para encontrar y corregir errores.